

# **AttendanceOps: A Multi-Tenant Attendance Management Platform**

*by*

**Md Selim Reza**

**x position**

**Mahdi Moniruzzaman**

**x position**

**Sakib Ahmed**

**x position**

*Team Project by Batch-6  
requirements for the degree of*

**SUMMER 2026**



© *Md Selim Reza, Mahdi Moniruzzaman, Sakib Ahmed*  
All rights reserved.

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| <b>Title</b>      | AttendanceOps: A Multi-Tenant Attendance Management Platform |
| <b>Author</b>     | Md Selim Reza, Mahdi Moniruzzaman, Sakib Ahmed               |
| <b>Student ID</b> | x position, x position, x position                           |

---

This document, *AttendanceOps: A Multi-Tenant Attendance Management Platform*, is an original work prepared by the AttendanceOps team, developed independently as a production-grade software product. It is not submitted in fulfillment of any academic requirement or degree. All third-party tools, libraries, and external references used in its design and development have been properly acknowledged, and all architecture, design decisions, and content described herein are the original work of the team unless otherwise credited..

---

**Md Selim Reza**  
x position

Department of Computer Science and Engineering  
Green University of Bangladesh

---

**Mahdi Moniruzzaman**  
x position

---

**Sakib Ahmed**  
x position

**Date:** July 13, 2026

## ABSTRACT

Attendance tracking for organizations running recurring online sessions, bootcamps, training programs, and cohort-based teams, still relies on manual spreadsheets and ad-hoc tools like WhatsApp or Discord, a workflow that is slow for coordinators, trivially easy to falsify through a shared code or a proxy response, and impossible to audit fairly once a dispute comes up. AttendanceOps is a multi-tenant SaaS architecture designed to close that gap by treating attendance as a trust problem first and a data-entry problem second. The platform is built around one global user identity with organization- and batch-scoped memberships, resolved through a hybrid RBAC and PBAC permission model so every action is checked against both role and instance scope. Fraud is addressed in two layers: a rotating, session-bound attendance code that removes the most common cheating vector without adding friction for the honest student, and an asynchronous, queue-based risk-scoring engine that flags suspicious submissions for human review instead of auto-rejecting them. A ticket-based informed/leave workflow, with a single level of appeal, gives every disputed absence a documented, auditable resolution path rather than an informal one. The system is specified end-to-end: a 23-model PostgreSQL schema managed through Prisma, an 86-endpoint REST API contract spanning 14 modules, a 12-module service-layer architecture with an explicit one-directional dependency rule, and a separate platform-administration control plane isolated on its own authentication strategy. Subscription limits are enforced through Redis atomic counters rather than application-level checks, closing a common race condition at the plan-limit boundary. The result is a design-complete, implementation-ready specification for a fraud-resistant, fully auditable attendance platform, with presence-heartbeat analytics, a dynamic report builder, and deeper platform observability scoped for later phases. **Keywords:** Multi-Tenant SaaS Architecture, RBAC/PBAC Permission

Model, Attendance Fraud Prevention, Prisma & PostgreSQL Schema Design, REST API Design, Audit Trail & Trust Systems

# Contents

|          |                                                        |          |
|----------|--------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>1</b> |
| 1.1      | Background of the Project . . . . .                    | 1        |
| 1.2      | Problem Statement . . . . .                            | 1        |
| 1.3      | Motivation . . . . .                                   | 2        |
| 1.4      | Project Objectives . . . . .                           | 2        |
| 1.4.1    | General Objective . . . . .                            | 2        |
| 1.4.2    | Specific Objectives . . . . .                          | 2        |
| 1.5      | Project Scope . . . . .                                | 3        |
| 1.5.1    | What the project will cover . . . . .                  | 3        |
| 1.5.2    | What the project will not cover . . . . .              | 5        |
| 1.6      | Project Deliverables . . . . .                         | 5        |
| 1.6.1    | Software Components . . . . .                          | 5        |
| 1.6.2    | Technical Documentation . . . . .                      | 6        |
| 1.6.3    | Testing and Deployment Assets . . . . .                | 7        |
| <b>2</b> | <b>System Analysis, Design and Implementation</b>      | <b>8</b> |
| 2.1      | Requirement Analysis . . . . .                         | 8        |
| 2.1.1    | Functional Requirements . . . . .                      | 8        |
| 2.1.2    | Non-functional Requirements . . . . .                  | 10       |
| 2.2      | System Architecture . . . . .                          | 11       |
| 2.3      | Database Design . . . . .                              | 13       |
| 2.3.1    | Identity & Organization Cluster . . . . .              | 14       |
| 2.3.2    | Subscription & Plan Cluster . . . . .                  | 15       |
| 2.3.3    | Batch & Session Cluster . . . . .                      | 15       |
| 2.3.4    | Attendance & Trust Cluster . . . . .                   | 16       |
| 2.3.5    | Cross-Cutting: Audit, Notification & Webhook . . . . . | 17       |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 2.3.6    | Platform Administration Cluster . . . . .               | 17        |
| 2.3.7    | Cascade and Restrict Rules . . . . .                    | 18        |
| 2.3.8    | Permission Resolution . . . . .                         | 19        |
| 2.4      | UML Diagrams . . . . .                                  | 20        |
| 2.4.1    | Use Case Diagram . . . . .                              | 20        |
| 2.4.2    | Activity Diagram / Process Flow Diagram . . . . .       | 21        |
| 2.4.3    | Sequence Diagram . . . . .                              | 23        |
| 2.4.4    | Class Diagram (if applicable) . . . . .                 | 25        |
| 2.4.5    | Data Flow Diagram (DFD) . . . . .                       | 26        |
| 2.4.5.1  | DFD Level 0 . . . . .                                   | 26        |
| 2.4.5.2  | DFD Level 1 . . . . .                                   | 27        |
| 2.5      | Security and Privacy Considerations . . . . .           | 28        |
| 2.6      | Tools and Technologies . . . . .                        | 30        |
| 2.6.1    | Development Environment and Tools . . . . .             | 30        |
| 2.6.2    | Programming Languages and Frameworks Used . . . . .     | 30        |
| 2.6.3    | Implementation Details by Module . . . . .              | 31        |
| 2.6.4    | Integration of Modules . . . . .                        | 32        |
| 2.6.5    | API / External Library Integration . . . . .            | 33        |
| 2.7      | Challenges Faced During Implementation . . . . .        | 33        |
| 2.8      | Engineering Considerations . . . . .                    | 34        |
| 2.8.1    | Societal Impacts of Engineering Solutions . . . . .     | 34        |
| 2.8.2    | Environment and Sustainability Considerations . . . . . | 35        |
| 2.8.3    | Ethical Considerations . . . . .                        | 35        |
| <b>3</b> | <b>Testing and Results Analysis</b>                     | <b>36</b> |
| 3.1      | Testing . . . . .                                       | 36        |
| 3.1.1    | Testing Strategies . . . . .                            | 36        |
| 3.1.1.1  | Test Case Scenarios . . . . .                           | 37        |
| 3.1.1.2  | Performance Testing (Target Benchmarks) . . . . .       | 38        |
| 3.1.2    | Comparison with Initial Requirements . . . . .          | 38        |
| 3.1.3    | Strengths and Weaknesses of the Design . . . . .        | 38        |
| 3.1.4    | Planned User Feedback Process . . . . .                 | 39        |

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>4</b> | <b>Conclusion and Future Work</b>           | <b>40</b> |
| 4.1      | Summary of Design Work Completed . . . . .  | 40        |
| 4.1.1    | Fulfillment of Project Objectives . . . . . | 40        |
| 4.1.2    | Limitations of the Current Design . . . . . | 41        |
| 4.1.3    | Recommendations for Enhancement . . . . .   | 42        |
| 4.1.4    | Future Scope of the Project . . . . .       | 42        |
|          | <b>References</b>                           | <b>44</b> |

# List of Figures

|      |                                                                      |    |
|------|----------------------------------------------------------------------|----|
| 2.1  | Entity-Relationship Diagram — Identity & Organization . . . . .      | 14 |
| 2.2  | Entity-Relationship Diagram — Batch & Session . . . . .              | 16 |
| 2.3  | Entity-Relationship Diagram — Attendance, Tickets & Import . . . . . | 16 |
| 2.4  | Entity-Relationship Diagram — Platform Administration . . . . .      | 17 |
| 2.5  | Use Case Diagram . . . . .                                           | 20 |
| 2.6  | Employer Activity Diagram . . . . .                                  | 21 |
| 2.7  | Job Seeker Activity Diagram . . . . .                                | 22 |
| 2.8  | Sequence Diagram . . . . .                                           | 23 |
| 2.9  | Sequence Diagram . . . . .                                           | 24 |
| 2.10 | Class Diagram – Core Entities . . . . .                              | 25 |
| 2.11 | Data Flow Diagram (DFD) Level 0 . . . . .                            | 26 |
| 2.12 | Data Flow Diagram (DFD) Level 1 . . . . .                            | 27 |



# 1. Introduction

## 1.1 Background of the Project

Organizations running online bootcamps, training programs, and coaching cohorts often rely on spreadsheets, shared codes, or chat messages to track attendance. As these programs grow, manual tracking becomes time-consuming, easy to manipulate, and difficult to audit.

AttendanceOps is a multi-tenant SaaS platform that automates the entire attendance workflow for online cohorts. It provides secure attendance submission with rotating codes, session management, manual corrections with mandatory reasons, an appeals process, and comprehensive audit logs for every action. Designed for multiple organizations, the platform ensures complete tenant isolation, role-based access, subscription management, and centralized administration, making attendance tracking accurate, transparent, and scalable.

## 1.2 Problem Statement

Existing attendance methods for online bootcamps and training programs rely heavily on manual processes such as spreadsheets, shared codes, and chat-based confirmations. These approaches are inefficient, vulnerable to fraudulent submissions, and lack accountability, making attendance records difficult to verify or dispute. Furthermore, administrators have limited visibility into attendance trends and participant engagement, preventing timely intervention. A secure, automated, and auditable attendance management system is needed to improve accuracy, reduce administrative effort, and provide reliable insights for decision-making.

## 1.3 Motivation

The people running these programs are already stretched thin, and manual attendance is exactly the kind of repetitive work a system should be handling instead of a human. At the same time, the informality of the current process is what makes it exploitable — a shared code or a proxy message costs nothing to fake and nothing to catch. AttendanceOps is motivated by closing both gaps at once: remove the manual burden, and make cheating meaningfully harder, without turning attendance into a chore for the student who's just trying to submit and get back to the session.

## 1.4 Project Objectives

### 1.4.1 General Objective

To build a multi-tenant attendance operations platform that automates attendance collection and verification for organizations running recurring online sessions, while giving every role — Admin, Mentor, CR, Student — a scoped view of exactly what they need, and giving the platform operator full visibility and control across every tenant on the system.

### 1.4.2 Specific Objectives

- To design a multi-tenant architecture that enables multiple organizations to use the platform while ensuring complete data isolation and role-based access control.
- To implement an identity and membership model that allows a single user to hold different roles across multiple organizations and batches with organization-scoped permissions.
- To develop a secure attendance submission mechanism using rotating, time-bound attendance codes that minimizes fraudulent submissions while maintaining a simple user experience.
- To build a session management module for creating, scheduling, and managing attendance sessions for different organizations and batches.

- To implement a manual attendance correction and appeals system that allows legitimate attendance disputes to be reviewed and resolved with proper justification.
- To develop passive fraud detection mechanisms, including device fingerprinting, duplicate-device detection, and impossible-timing analysis, without delaying attendance submission.
- To maintain a comprehensive audit logging system that records every manual attendance action, including who performed it, when it occurred, and the reason behind it.
- To enforce subscription plan limits (e.g., maximum batches, students, and employees) using atomic counters to ensure consistency under concurrent requests.
- To provide role-based dashboards and reporting features, including customizable reports for analyzing attendance trends and identifying at-risk students.
- To develop a dedicated platform administration module that enables SaaS operators to manage organizations, monitor system health, enforce subscription plans, and investigate fraudulent activities independently of tenant administration.

## 1.5 Project Scope

### 1.5.1 What the project will cover

#### 1. Identity and Membership Management:

- Single global user identity with organization-scoped memberships.
- Role-based access control for Admin, Mentor, CR, and Student.
- Support for users holding different roles across multiple organizations.

#### 2. Organization and Batch Management:

- Organization profile and configurable settings.

- Employee invitation, role assignment, activation, and deactivation.
- Batch creation, management, archiving, and subscription plan enforcement.

### **3. Student Management:**

- Manual student registration and bulk CSV import.
- Student profile management and batch enrollment history.

### **4. Session and Attendance Management:**

- One-time and recurring session scheduling with attendance lifecycle management.
- Secure attendance submission using rotating, time-bound attendance codes.
- Student self-attendance submission and CR manual attendance marking with mandatory justification.
- Presence tracking to monitor student engagement during online sessions.

### **5. Fraud Prevention and Audit Logging:**

- Fraud prevention using rotating codes and duplicate submission detection.
- Passive fraud detection through device, IP, and timing analysis.
- Comprehensive audit logs for all attendance-related administrative actions.

### **6. Tickets, Appeals, and Reporting:**

- Leave requests, attendance appeals, and approval workflow.
- Attendance reports, analytics dashboards, dynamic report generation, and CSV/Excel export.

### **7. Notifications and Platform Administration:**

- Email, in-app, and Discord webhook notifications.
- Dedicated platform administration for tenant management, subscription control, system monitoring, and platform-wide audit logging.

## 1.5.2 What the project will not cover

### 1. Biometric or Location-based Attendance:

The system will not include face recognition, fingerprint authentication, GPS verification, or other biometric attendance mechanisms.

### 2. Video Conference Integration:

The platform will not verify attendance through direct integration with Google Meet, Zoom, or similar video conferencing APIs.

### 3. Native Mobile Applications:

The project will provide a responsive web application only and will not include dedicated Android or iOS applications.

### 4. AI-based Fraud Detection and LMS Features:

The system will use rule-based fraud detection and will not implement machine learning models, assignment management, quizzes, or other Learning Management System (LMS) functionalities.

### 5. Billing and Advanced Platform Features:

The project will not include payment gateway integration, subscription billing workflows, or a full Discord bot. Plan management and webhook notifications are included, while payment processing is reserved for future development.

## 1.6 Project Deliverables

As output, the project yields the full set of software artifacts — code, configuration, architectural models, and documentation — needed to take AttendanceOps from design to a running, testable system.

### 1.6.1 Software Components

- **Full Web Application:** A responsive web system with role-scoped dashboards for Admin, Mentor, CR, and Student, plus a separate Platform Admin surface for the SaaS operator.

- **Full Source Code:** Complete monorepo source (pnpm workspaces) covering the NestJS backend (apps/api), the frontend (apps/web), and shared types (packages/shared-types), with Docker Compose configuration for local Postgres/Redis and environment setup.
- **RESTful API:** A stable, permission-guarded API (86 routes across Auth, Organization, Batch, Session, Attendance, Ticket, Fraud, Reports, Notifications, and Platform Admin modules) covering authentication, attendance submission and verification, fraud review, and ticket/appeal workflows.
- **Fraud Detection Module:** Rule-based prevention (rotating attendance codes, duplicate-submission blocking) and async heuristic risk scoring (device fingerprinting, duplicate-device and impossible-timing detection) feeding a manual review queue.

### 1.6.2 Technical Documentation

- **Product Requirement Document (PRD):** The complete requirement specification — identity model, role responsibilities, functional modules, and non-functional requirements — serving as the single source of truth for scope.
- **Diagram Set (complete series of system-modeling diagrams):**
  - **Class/Schema Diagram:** Prisma schema structure — core entities (User, Membership, BatchMembership, Batch, Session, Attendance, Ticket) and their relationships.
  - **Activity Diagrams:** Workflows for each actor — Student submission, CR window control, Mentor ticket resolution, Admin plan/employee management.
  - **Sequence Diagrams:** Key flows such as attendance submission, ticket appeal resolution, and fraud review.
- **API Contract / Documentation:** Route-by-route specification (method, path, required permission, request/response shape) for all 86 endpoints, later superseded by an auto-generated Swagger contract at /api/docs once controllers exist.

- **Permission Matrix:** The single source of truth mapping every action string to the roles allowed to perform it, used directly by both backend guards and frontend conditional rendering.

### 1.6.3 Testing and Deployment Assets

- **Test Suite & Report:** Unit tests for permission-check logic and attendance-submission uniqueness/window enforcement (the two highest-risk areas for silent bugs), integration tests against Postgres/Redis, and a summary report of coverage and results.
- **Onboarding & Deployment Guide:** Step-by-step local setup (clone to running environment in under 20 minutes), environment variable reference, and deployment configuration for the target hosting providers.

## 2. System Analysis, Design and Implementation

### 2.1 Requirement Analysis

#### 2.1.1 Functional Requirements

##### 1. User Authentication and Membership Management:

**Registration and Login:** On signup, the system creates a global User identity and auto-creates an Organization with an ADMIN membership for that user — no separate “create organization” step. Login issues a JWT carrying the active organization context.

**Multi-Org Membership:** A single user can hold independent memberships across multiple organizations (e.g. Student in one org, Mentor in another), and can switch active organization context without logging out.

**Role Assignment:** Admins can invite employees, assign roles (Admin, Mentor), and activate/suspend memberships. Mentors assign and revoke the CR flag on students within their own batch.

##### 2. Organization and Batch Management:

**Organization Settings:** Admins manage organization profile (name, logo, timezone) and settings (default attendance duration, late threshold).

**Batch Management:** Admins and Mentors can create, update, and archive batches, subject to the organization’s plan limits (maxBatches). Each batch supports Mentor and CR assignment, with CR reassignment possible at any time.

**Student Management:** Students can be added individually or via bulk CSV import, with per-row validation and a result report for invalid rows, subject to the plan’s maxStudents limit.



### **3. Session and Attendance Management:**

**Session Scheduling:** Mentors can create one-time or recurring sessions (daily / weekly / custom patterns), with lifecycle states from Scheduled through Ended or Cancelled.

**Attendance Window Control:** Only a CR or Mentor can open and close a session's attendance window; students may submit only while the window is open.

**Attendance Submission:** Students submit attendance with a one-tap, rotating session-bound code. Status (Present/Late) is derived automatically from submission time against a configurable threshold. CRs can manually mark attendance during the open window with a mandatory reason.

**Tickets and Appeals:** Students can raise an informed / leave ticket before a session ends. A CR decision can be appealed once to the Mentor, whose decision is final; every step is logged with a required reason.

### **4. Fraud Detection and Presence Tracking:**

**Fraud Prevention:** Rotating, time-bound attendance codes and hard duplicate-submission blocking prevent the most common cheating methods without adding steps for honest students.

**Fraud Detection:** The system asynchronously captures device, IP, and timing signals at submission, flags duplicate-device and impossible-timing patterns, and produces a heuristic risk score for CR/Mentor review — never auto-rejecting a submission.

**Presence Tracking:** A client-side heartbeat during the attendance window builds a per-student presence timeline, classifying gap patterns for human review rather than automatic penalty.

### **5. Reporting, Audit and Notifications:**

**Audit Logging:** Every manual action (attendance edit, ticket decision, role change) is logged with actor, reason, timestamp, and before/after value.

**Reporting:** Role-scoped standard reports (student, session, batch, organization), a filter-driven analytics engine, and a dynamic report builder with CSV/Excel/PDF export.

**Notifications:** Email and in-app notifications for session reminders, attendance status, and ticket decisions, plus Discord webhook delivery for session announcements.

### 2.1.2 Non-functional Requirements

#### 1. Performance:

**Latency:** Any user action should complete in under 2 seconds under normal load, including attendance submission and search/filter operations.

**Concurrency:** The system must support at least 200 concurrent users without a significant performance drop, including simultaneous attendance submissions at the moment a window opens — the platform’s highest-traffic scenario.

#### 2. Usability:

**Responsiveness:** The platform must be mobile-responsive, since students frequently submit attendance from a phone during a live session.

**Ease of Use:** Attendance submission must remain a one-tap action for students regardless of the fraud-prevention logic running behind it — no added friction for the honest user.

**Role Clarity:** Each role (Admin, Mentor, CR, Student) should see only the dashboard widgets and actions relevant to their scope, so the interface doesn’t force users to parse permissions they don’t have.

**Accessibility:** The system should meet minimum WCAG standards, including screen reader and keyboard navigation support.

#### 3. Reliability and Availability:

**Uptime:** The platform targets 99.9% uptime, with scheduled maintenance windows communicated in advance via platform-wide broadcast.

**Data Integrity:** No attendance or audit data may be lost on failure — automated backups and replication for critical records. Background job failure rate (CSV imports, async fraud scoring) must stay under 1%.

**Error Handling:** Errors surface as clear, human-readable messages (e.g. “Attendance is not currently open for this session”) rather than raw stack traces or generic failures.

#### 4. Security:

**Authentication:** JWT-based authentication with bcrypt password hashing. Platform Admin identity uses a separate token audience/secret entirely isolated from tenant-side authentication.

**Authorization:** Every guarded route enforces RBAC combined with scope checks (PBAC) — no permission check may run without an organization (and where relevant, batch) scope.

**Data Protection:** All personal data is encrypted at rest; all client-server communication runs over HTTPS. Platform Admin impersonation of a tenant user must be logged and visibly flagged to the affected org.

#### 5. Scalability:

**Horizontal/Vertical Scale:** The system must scale in either direction as tenant count and session volume grow, without requiring architectural rework.

**Plan-Limit Concurrency:** Limit checks (`maxBatches`, `maxStudents`, `maxEmployees`) must use Redis atomic counters rather than application-level count-then-create logic, to prevent race conditions when multiple requests land near a plan's cap simultaneously.

## 2.2 System Architecture

AttendanceOps is built as a layered, service-oriented architecture, with a hard rule that dependencies only ever point in one direction. A presentation layer (the web frontend, built in React/Next.js) talks to an application layer of independent backend modules over a REST API, and each module owns a specific slice of the schema — no module reaches into another module's tables directly. The data layer sits underneath, split between PostgreSQL for relational data (organizations, batches, sessions, attendance, tickets) and Redis for high-throughput needs like atomic plan-limit counters and BullMQ job queues.

The application layer is organized into twelve modules, each mapping to a folder in the backend codebase:

```

AuthModule
  used by every other module (guards, decorators)
  |
  +-- OrgModule
  |   |
  |   +-- BatchModule
  |       |
  |       +-- SessionModule
  |           |
  |           +-- AttendanceModule
  |               |
  |               +-- TicketModule
  |               +-- FraudModule
  |   +-- SubscriptionModule
  |
  +-- StudentImportModule
  |   (depends on OrgModule, BatchModule)
  |
  +-- ReportsModule
  |   (depends on AttendanceModule, TicketModule,
  |       FraudModule -- read-only)
  |
  +-- NotificationModule
  |   (depends on nothing upstream --
  |       consumed by everyone)
  |
  +-- PlatformAdminModule
  |   (separate, cross-cutting -- depends on
  |       OrgModule, SubscriptionModule read
  |       access only)

```

**AuthModule** sits at the foundation and resolves who the user is and which organization context they’re acting in — it does not decide whether an action is permitted; that’s a separate, reusable `PermissionsService` that every module calls independently, checked against the action strings defined in the Permission Matrix.

**OrgModule** is the only module allowed to write to `Membership`; **BatchModule** is the only one allowed to write to `BatchMembership`, including the CR flag. This “single writer per table” rule is what keeps role and scope data trustworthy — any other module needing that information reads it through the owning module’s exposed service methods, never through its own repository query.

**SubscriptionModule** centralizes every plan-limit check (`maxBatches`, `maxStudents`, `maxEmployees`) behind one `checkLimit(orgId, resource)` method, backed by Redis atomic counters. This keeps limit enforcement from being duplicated — and potentially implemented inconsistently — across the Batch, Org, and Student-import creation flows.

**SessionModule** owns the rotating attendance-code logic and session lifecycle transitions. **AttendanceModule** and **TicketModule** enforce their own business rules (one submission per student per session, the informed/leave state machine) but never write session-lifecycle events themselves — that responsibility stays with **SessionModule** alone.

**FraudModule** and **StudentImportModule** both run as BullMQ queue consumers rather than executing inline during a request. This is deliberate: fraud scoring and large CSV imports are exactly the kind of work that would otherwise turn a single “submit attendance” or “upload student list” click into a slow, blocking request — running them asynchronously is what keeps the one-tap submission promise intact for the student.

**ReportsModule** owns no tables of its own; it is a purely read-side module composing queries across Attendance, Session, Batch, and Ticket data, which keeps it safe to eventually point at a read replica without touching any other module’s write path.

**PlatformAdminModule** is deliberately isolated from the rest of the system. It is not consumed by any tenant-facing module, and it authenticates through a completely separate token strategy (different secret and audience) so that a bug in the tenant-side auth guard can never accidentally grant platform-level access, and vice versa.

## 2.3 Database Design

The schema is built around one rule, stated in the project’s schema guide: *every record belongs to an Organization from day one*, even though v1 only ever has one Admin per org. Nothing points at User directly except Membership — this single decision is what lets a solo-founder org today become a real multi-tenant org later without a data migration.

The schema is organized into five logical clusters, each mapping to the module that owns it (Section 3.3): Identity & Organization, Subscription & Plan, Batch & Session, Attendance & Trust, and Platform Administration.

### 2.3.1 Identity & Organization Cluster

- **User** — global identity, one row per human: `id`, `email`, `name`, `passwordHash` (nullable, for OAuth users), `authProvider`, `deletedAt`. Never carries a role, an org, or a batch.
- **Organization** — `id`, `name`, `logoUrl`, `timezone`, `deletedAt`. Auto-created silently at signup, with no dedicated “create org” screen in v1.
- **OrganizationSettings** — `id`, `organizationId` (FK, 1:1), `defaultAttendanceDurationMins`, `lateThresholdMins`
- **Membership** — `id`, `userId` (FK), `organizationId` (FK), `role` (ADMIN|MENTOR|STUDENT), `status` (ACTIVE|INVITED|SUSPENDED), `invitedByMembershipId`, `inviteToken`, `inviteExpiresAt`, `deletedAt`
- **BatchMembership** — `id`, `membershipId` (FK), `batchId` (FK), `isCR`, `assignedAt`, `revokedAt`

The unique constraint on Membership is `@@unique([userId, organizationId, role])`, deliberately **not** (`userId, organizationId`). A founder who is ADMIN and also personally mentors a batch needs two Membership rows in the same org — with the narrower unique key, that would be structurally impossible without a schema change. Under this key, it’s just another row.

BatchMembership is where the CR flag lives (`isCR`), not on the student’s core profile — CR is a permission flag on a specific batch enrollment, not a separate account type. Reassigning CR status is revoking one row (`revokedAt`) and granting another, so batch history is never lost.

Figure 2.1: Entity-Relationship Diagram — Identity & Organization

### 2.3.2 Subscription & Plan Cluster

- **Plan** — id, name, maxBatches, maxStudents, maxEmployees, priceCents, featureFlags (JSON)
- **Subscription** — id, organizationId (FK, 1:1), planId (FK), status (SubscriptionStatus), currentPeriodEnd, overrideMaxBatches, overrideMaxStudents, overrideMaxEmployees, overrideReason, overriddenByPlatformAdminId (FK, nullable)

Plan limits are data, never hardcoded in application logic — enforcement checks `Subscription.plan.maxBatches` (or the `override*` field, when non-null, which always takes precedence) against a live count, using a Redis atomic counter at the point of creation to prevent two simultaneous batch-creation requests from both slipping past the limit.

### 2.3.3 Batch & Session Cluster

- **Batch** — id, organizationId (FK), name, startDate, endDate, capacity, defaultMeetLink, isArchived, lateThresholdMinsOverride, attendanceDurationMinsOverride
- **SessionTemplate** — id, batchId (FK), title, description, recurrenceRule (JSON, e.g. `{"days": ["SUN", "TUE", "THU"], "time": "21:00"}`), durationMins, meetLink, isActive
- **Session** — id, batchId (FK), templateId (FK, nullable), title, scheduledStart, scheduledEnd, type, status, attendanceOpenedAt, attendanceOpenedById (FK to Membership), attendanceClosedAt, attendanceClosedById (FK), currentCode, codeRotatedAt
- **SessionActivityEvent** — id, sessionId (FK), type (STARTED | ATTENDANCE\_OPENED | STUDENT\_...), actorMembershipId (FK, nullable), metadata (JSON)

A template defines the recurrence pattern; concrete `Session` rows are generated from it by a scheduled job rather than created by hand each day. One-time sessions skip the template entirely. `attendanceOpenedById` and `attendanceClosedById` are real `Membership` foreign keys, not bare strings — the schema itself guarantees you always know exactly who opened or closed a given session's window.

Figure 2.2: Entity-Relationship Diagram — Batch & Session

### 2.3.4 Attendance & Trust Cluster

- **Attendance** — `id`, `sessionId` (FK), `studentBatchMembershipId` (FK), `status` (`AttendanceStatus`), `method` (`SELF_SUBMITTED` | `MANUAL`), `submittedAt`, `markedByMembershipId` (FK, nullable), `manualReason`, `ipAddress`, `deviceFingerprint`, `userAgent`, `riskScore`, `isFlagged`, `reviewedByMembershipId` (FK, nullable), `reviewedAt`, `reviewOutcome` (null until reviewed)
- **PresenceHeartbeat** — `id`, `attendanceId` (FK), `userId` (FK), `pingAt`
- **Ticket** — `id`, `sessionId` (FK), `attendanceId` (FK), `studentId` (FK), `reason`, `status` (`TicketStatus`), `crDecisionById` (FK), `crDecisionReason`, `crDecisionAt`, `mentorDecisionById` (FK), `mentorDecisionReason`, `mentorDecisionAt`
- **StudentImportJob** / **StudentImportRow** — a parent job (`status`, `totalRows`, `successCount`, `failureCount`) with one child row per CSV line (`rowNumber`, `rowData` JSON, `status`, `errorMessage`), so an import produces a row-by-row report instead of a single pass/fail

`@@unique([sessionId, studentBatchMembershipId])` on `Attendance` is the hard, database-level guarantee against duplicate submissions — it backs up the application-layer check rather than replacing it. `method` distinguishes a student’s own tap from a CR’s manual override, which matters because manual entries are excluded from fraud scoring but retained for CR-activity auditing. `riskScore` and `isFlagged` are set asynchronously by the fraud-scoring job and never block a submission. `crDecisionBy` and `mentorDecisionBy` are separate `Membership` relations on `Ticket` — not one shared field — because a ticket can pass through both decisions at different times and both need to be preserved, not overwritten.

Figure 2.3: Entity-Relationship Diagram — Attendance, Tickets & Import



### 2.3.5 Cross-Cutting: Audit, Notification & Webhook

- **AuditLog** — id, organizationId (FK, nullable), actorUserId (FK, nullable), action (e.g. "attendance.manual\_mark"), targetType, targetId, beforeValue (JSON), afterValue (JSON), reason
- **Notification** — id, userId (FK), organizationId (FK, nullable), type, channel, payload (JSON), isRead, deletedAt
- **WebhookConfig** — id, organizationId (FK), batchId (FK, nullable — null means org-wide), url, events (String array), isActive

AuditLog has no single owning module — every module writes to it on any mutating action, which is why it sits outside the five main clusters rather than inside one of them.

### 2.3.6 Platform Administration Cluster

- **PlatformAdmin** — id, email, name, passwordHash, role (SUPPORT|PLATFORM\_OWNER)
- **PlatformAuditLog** — id, platformAdminId (FK), action (e.g. "org.suspend", "plan.override"), targetOrgId, metadata (JSON), reason
- **ImpersonationSession** — id, platformAdminId (FK), targetUserId (FK), targetOrganizationId (FK, nullable), reason (required), startedAt, endedAt

This cluster exists entirely outside the Membership/Organization model — PlatformAdmin is never a User row and never holds a Membership, so platform staff can never be represented as a tenant-side role. PlatformAuditLog is a fully separate table from tenant-side AuditLog, so a platform admin's actions on a customer's org are never queryable or deletable through tenant-facing code paths.

Figure 2.4: Entity-Relationship Diagram — Platform Administration

### 2.3.7 Cascade and Restrict Rules

Delete behavior is deliberately not uniform across the schema — the guiding principle is that anything representing “*this thing happened*” must never silently disappear because something upstream was cleaned up, while anything representing “*this thing currently exists*” can safely cascade, since the application layer soft-deletes or archives those records anyway.

| Relation                            |   | Behavior        | Reason                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------|---|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Membership<br>User/Organization     | → | Cascade         | Both are soft-deleted ( <code>deletedAt</code> ) in practice; cascade is a safety net for the rare true hard-delete.                                                                                                                                                                                                                                  |
| BatchMembership<br>Membership/Batch | → | Cascade         | Same reasoning — but see the row below for what actually prevents accidental data loss.                                                                                                                                                                                                                                                               |
| Attendance<br>BatchMembership       | → | <b>Restrict</b> | If a <code>BatchMembership</code> has any <code>Attendance</code> rows, Postgres refuses to delete it, even if something upstream tries to cascade into it. This is what physically prevents “remove a student from a batch” from silently erasing their attendance history — to remove a student, set <code>revokedAt</code> , never delete the row. |
| AuditLog<br>Organization/User       | → | SetNull         | The audit entry survives even if its subject is later deleted — the historical record stays, just with a null reference instead of a broken one.                                                                                                                                                                                                      |
| Attendance → Session                |   | Cascade         | Sessions are practically never hard-deleted (they’re cancelled via status), so this is a safety net rather than an everyday path.                                                                                                                                                                                                                     |

`Attendance`, `Ticket`, `AuditLog`, `PlatformAuditLog`, and `SessionActivityEvent` deliberately have no `deletedAt` field — these are historical records the product’s entire audit layer depends on being permanent. Where a correction is needed, the pattern is a new row referencing the old one (exactly how a ticket appeal adds a new resolution rather than erasing the original rejection), not a soft delete. `Notification` is the one exception, since a user clearing their notification list is a normal UX action with no audit implications.

### 2.3.8 Permission Resolution

Every permission check in the system resolves through Membership — never a shortcut such as comparing organization IDs directly. The check filters active, non-deleted memberships for the acting user in the target organization; an ADMIN role grants implicit access to every batch in v1, while MENTOR and STUDENT roles additionally require an active, non-revoked BatchMembership for the specific batch in scope, with the isCR flag further unlocking CR-level actions for a student. Full detail is in 05-Permission-Matrix.md.



### 2.4.2 Activity Diagram / Process Flow Diagram

The Employer Activity Diagram illustrates the workflow of a recruiter, starting from Authentication. Once logged in, the employer has two main paths: Job Management and Applicant Review. They can post new job openings. During the process, they can review incoming applications, filter candidates, and update the status of the application. The flow concludes when the recruiter logs out, ensuring a secure session throughout.

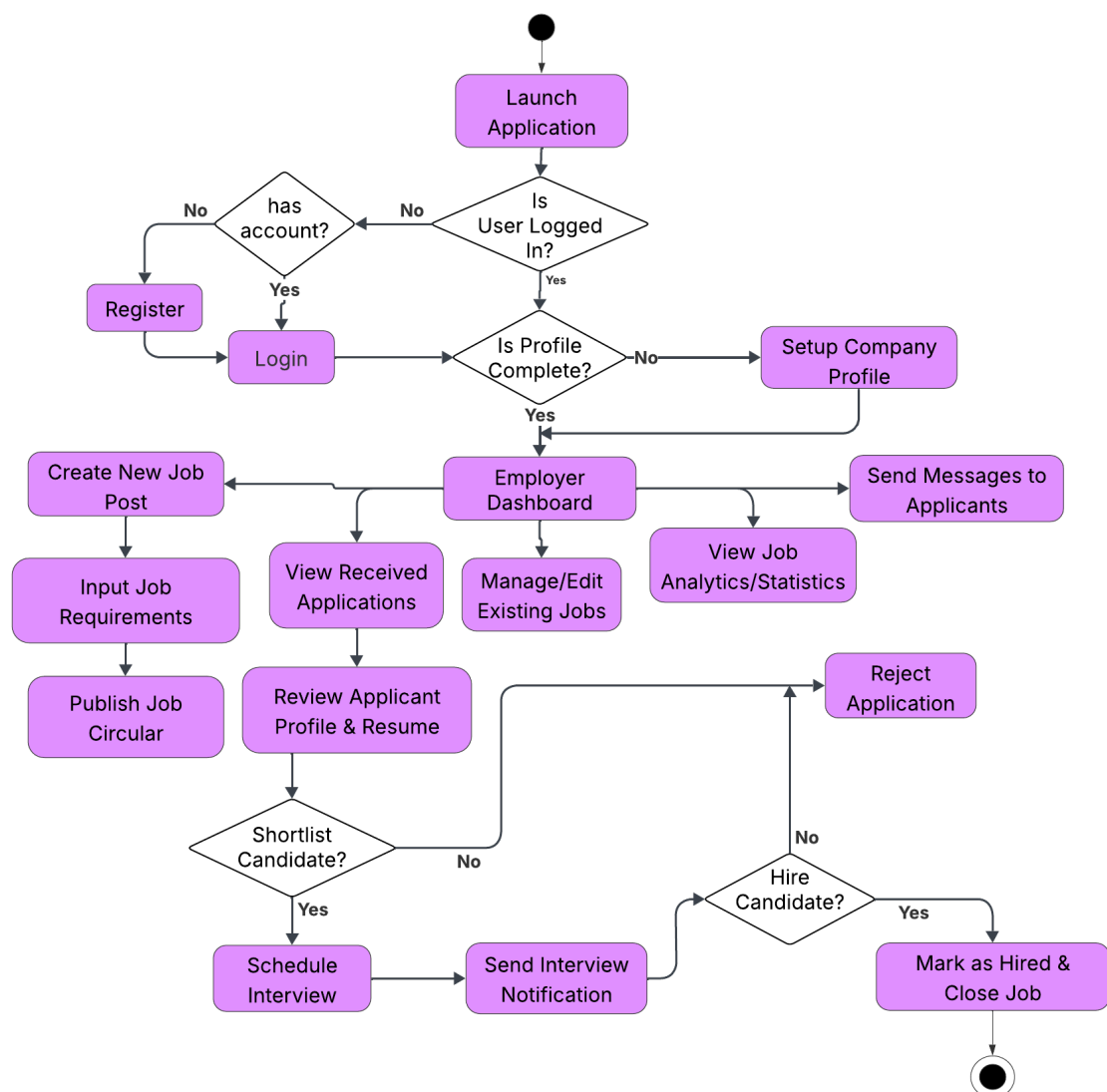


Figure 2.6: Employer Activity Diagram

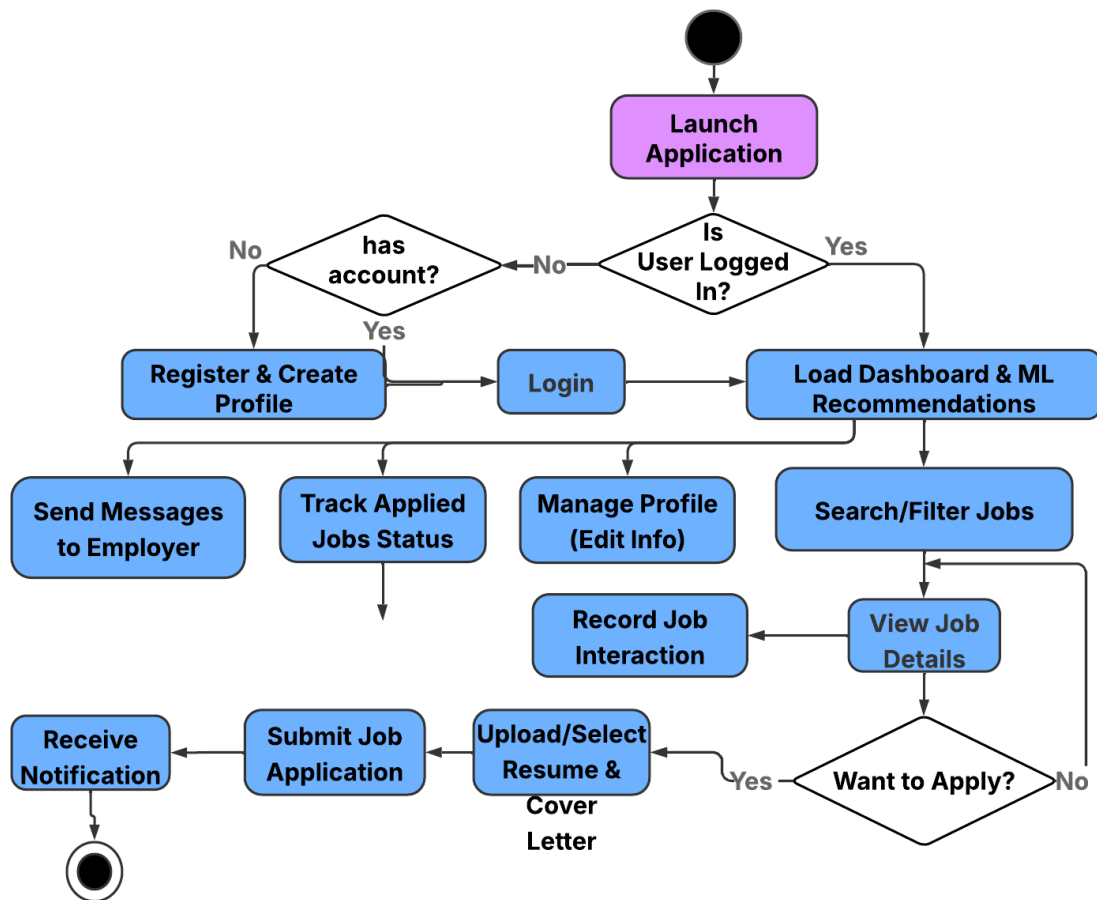


Figure 2.7: Job Seeker Activity Diagram

The Job Seeker Activity Diagram outlines the journey of a candidate, beginning with Registration and Login. Once authenticated, the user can Build their Profile, which our Gemini AI analyzes for semantic matching. The system then generates a Personalized Job Feed. From here, the seeker can search for jobs, view details, and Submit Applications. The flow ensures that every action—from profile update to final application—is logged in the MongoDB database for real-time tracking.

### 2.4.3 Sequence Diagram

This sequence diagram illustrates the step-by-step interactions between a Job Seeker, Employer, and Admin as they engage with the system's UI, Backend, and Database for processes like registration, login, and profile management. It highlights the end-to-end data flow, ensuring that user requests are validated and correctly stored within the job portal's architecture.

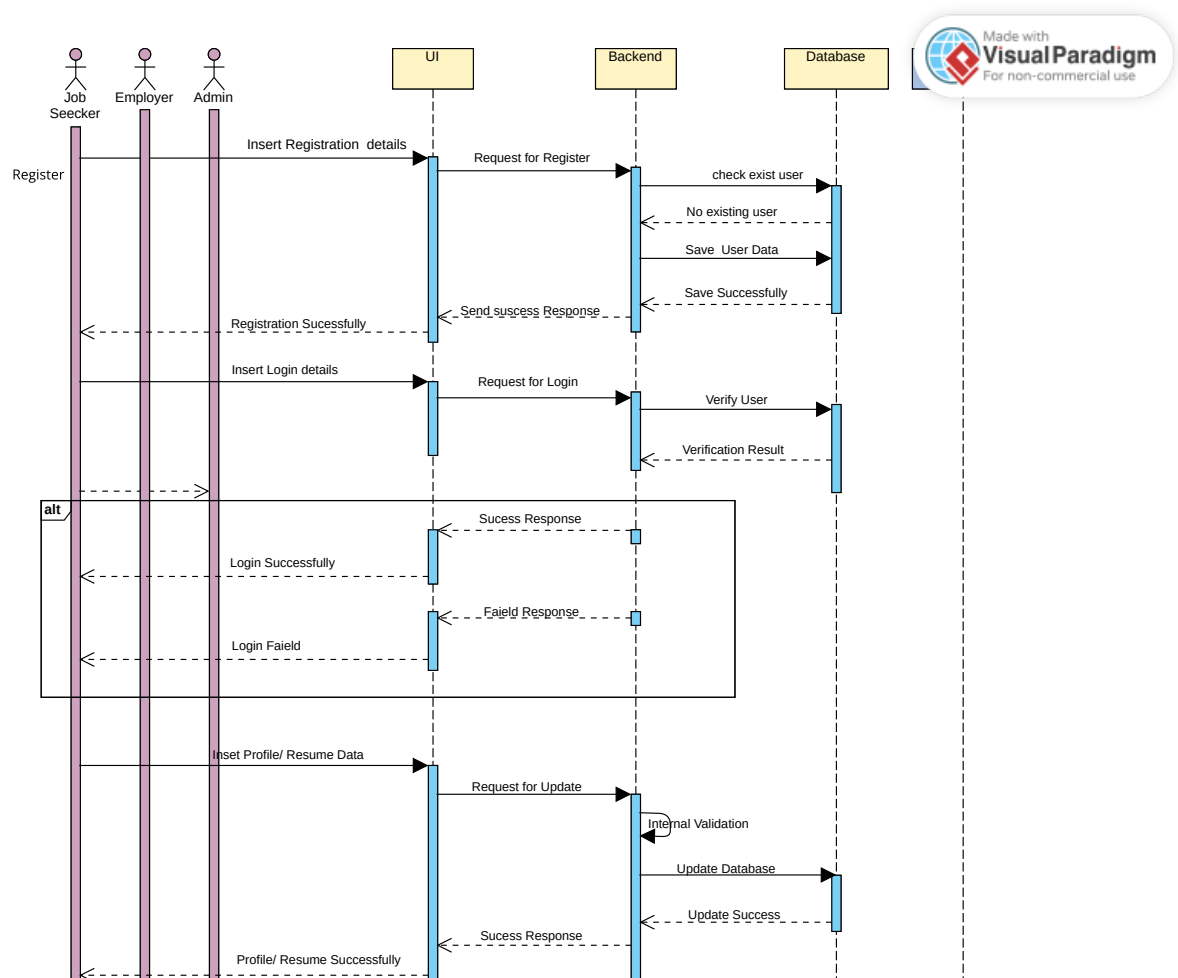


Figure 2.8: Sequence Diagram

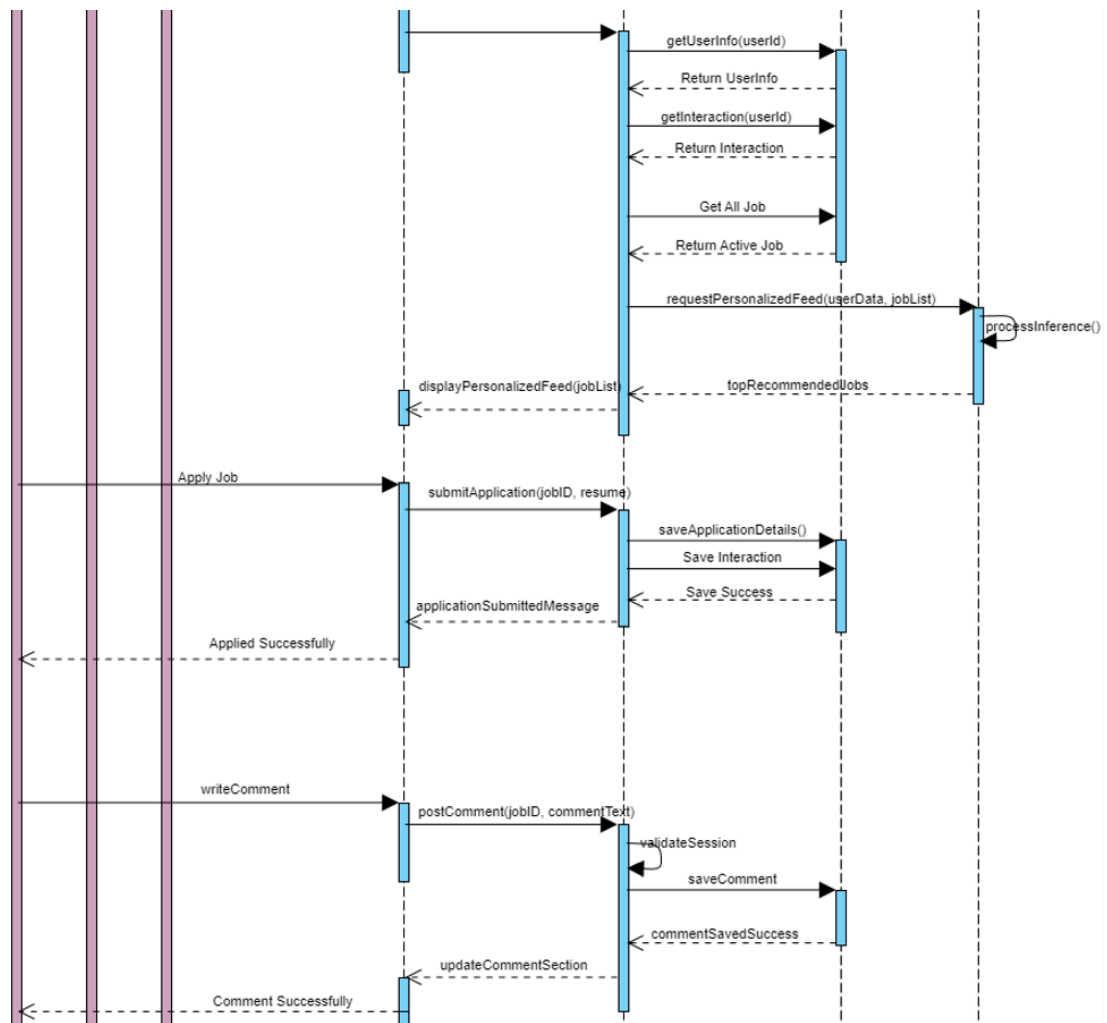


Figure 2.9: Sequence Diagram

This sequence diagram depicts the interaction flow between users and a job portal system for registration, login, and profile updates. It maps how requests move from the interface to the backend and database to complete these tasks.



## 2.4.4 Class Diagram (if applicable)

Our Class Diagram represents the static structure of the system, showcasing the attributes and methods of our primary entities: User, Job, and Application. It illustrates the relationships between these classes—for instance, how a User can have multiple Applications, and how each Job is linked to a specific Recruiter. This ensures a clear blueprint for our database schema and object-oriented logic.

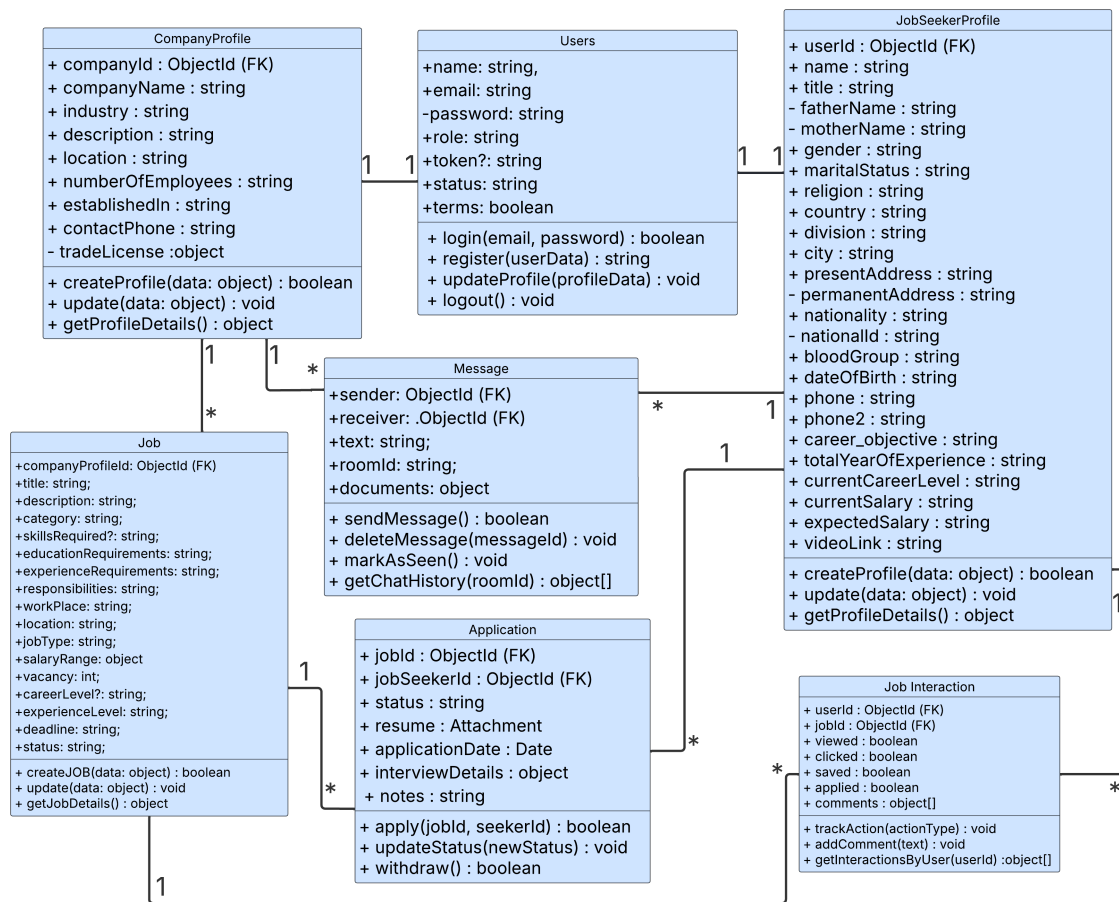
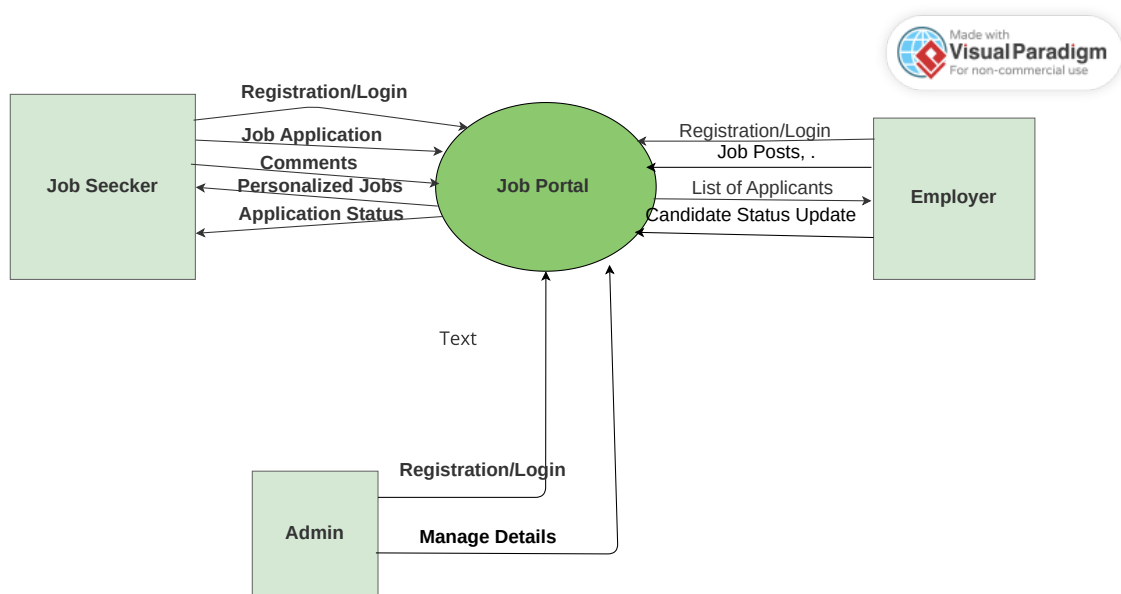


Figure 2.10: Class Diagram – Core Entities

## 2.4.5 Data Flow Diagram (DFD)

### 2.4.5.1 DFD Level 0

Our DFD Level 0, or Context Diagram, defines the overall scope of the system. It shows the central Job Portal Process interacting with our three external entities: Job Seeker, Recruiter, and Admin. It illustrates how data—like profiles and job posts—flows into the system and how processed information, such as job alerts and analytics, flows back to the users.



DFD Level 0

Figure 2.11: Data Flow Diagram (DFD) Level 0

### 2.4.5.2 DFD Level 1

DFD Level 1 breaks down the main system into detailed sub-processes like User Management, Job Matching, and Application Tracking. It shows how data flows between these processes and our Data Stores, such as the Job and User databases. This provides a clear view of the internal logic and how the AI Engine interacts with the data to generate results.

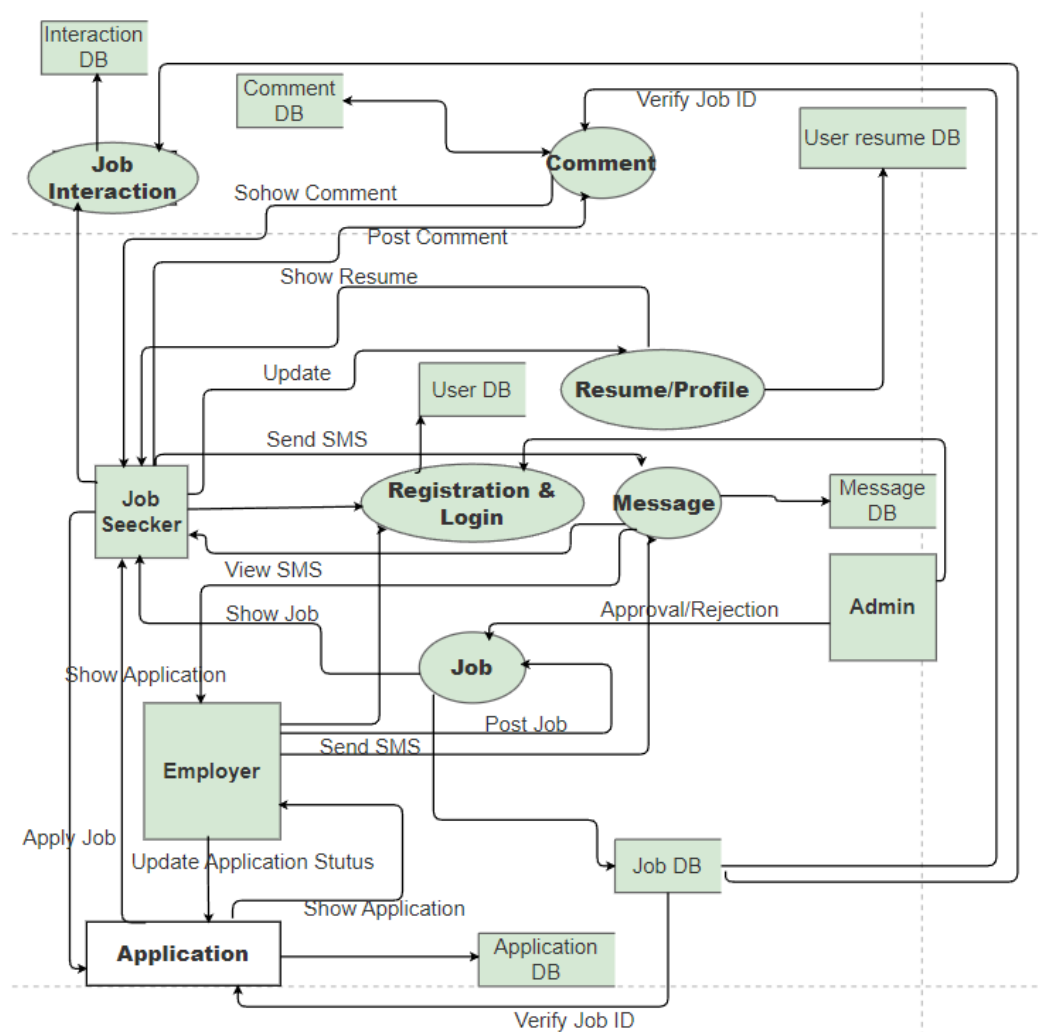


Figure 2.12: Data Flow Diagram (DFD) Level 1

## 2.5 Security and Privacy Considerations

AttendanceOps handles two categories of sensitive data that shape most of the security design: personal identity information (names, emails, student records) and behavioral/device data collected for fraud detection (IP addresses, device fingerprints, presence timelines). Both are treated as data that needs protecting from unauthorized access, not just from outside attackers but from over-broad access within the platform itself.

**Authentication.** User authentication runs on JWTs, with passwords hashed via bcrypt before storage — plaintext passwords are never written to the database under any code path. Platform Admin authentication is deliberately isolated from tenant authentication: a separate token secret and audience, issued through a separate login endpoint, so that a bug in the tenant-side auth guard can never be exploited to gain platform-level access, and a compromised tenant account can never escalate into platform control.

**Authorization.** Every guarded route enforces both RBAC and scope-based access control together — a permission check verifies not just that a role is generally allowed to perform an action, but that the specific organization (and, where relevant, batch) is one the acting user actually has an active membership in. A permission check that skips organization scope is treated as a bug during code review, never as an acceptable shortcut, since it's exactly the kind of gap that lets a Mentor from Org A see data from Org B.

**Data Protection.** Personal data is encrypted at rest, and all client-server communication runs over HTTPS. Fraud-detection signals (IP address, device fingerprint, user agent) are collected only at the point of attendance submission and are visible solely to CR/Mentor/Admin roles reviewing a flagged record — a student never sees another student's fraud data, and the raw signals are never exposed in any student-facing view.

**Audit Trail as a Security Control.** Every manual action that changes state — a manual attendance mark, a ticket decision, a role change, a CR reassignment — is logged with who performed it, when, why, and the before/after value. This isn't just for dispute resolution; it's a deterrent. A CR who knows their manual marks are individually attributed and reviewable behaves differently than one working inside an unaudited system. The same principle applies at the platform level: every

Platform Admin action is written to a separate, tenant-inaccessible audit log, so platform staff can't quietly edit a customer's data without a trace.

**Impersonation Controls.** Platform Admin impersonation — used for support, to view an org's environment as a specific user and debug a reported issue — requires a mandatory reason at the point of starting the session, and the session itself is time-boxed and logged with start and end timestamps. The impersonated organization can see afterward that an impersonation occurred, when, and why. Impersonation without this trail is treated as a trust failure in the product, not an acceptable convenience.

**Fraud-Detection Boundaries.** The fraud system is deliberately built to never block or delay a legitimate submission — all risk scoring runs asynchronously after the fact. This is a security design choice as much as a UX one: a system that can reject an honest student's submission based on a false-positive heuristic creates an incentive to weaken the heuristic, which in turn weakens fraud resistance for everyone. Keeping detection strictly post-submission and human-reviewed avoids that trade-off.

**Data Retention and Deletion.** Attendance records, tickets, and audit logs are never hard-deleted — they're the historical evidence the product's trust model depends on. Where a student needs to be removed from a batch, the system revokes their `BatchMembership` rather than deleting it, which the database itself enforces by refusing to delete any membership row that has associated attendance history. Org-level data export and deletion workflows exist for compliance and offboarding, but are built to preserve audit-log retention requirements even after a tenant's account is closed.

**Plan-Limit Integrity.** While not a conventional security concern, enforcing subscription plan limits (`maxBatches`, `maxStudents`, `maxEmployees`) through Redis atomic counters rather than application-level count-then-create logic closes a race-condition class of bug where two simultaneous requests near a plan's cap could both succeed, silently letting a tenant exceed what they're paying for.

## 2.6 Tools and Technologies

### 2.6.1 Development Environment and Tools

**Hardware Environment.** A processor in the Intel Core i5 / AMD Ryzen 5 class (or better) handles the day-to-day load of running the NestJS API, a Postgres/Redis Docker stack, and the frontend dev server simultaneously without noticeable lag. 8 GB of RAM is the practical minimum for keeping an IDE, multiple terminal sessions, and two Docker containers running at once; 16 GB is more comfortable once integration tests are running alongside everything else. An SSD matters more here than raw CPU speed, since Prisma migrations, Postgres writes, and ‘pnpm install’ across a monorepo are all disk-bound operations.

**Software Environment.** Development happens on Windows 10/11 or macOS day to day, with Ubuntu Linux used for backend testing and as the production target, since that’s what the deployment environment actually runs. The core editor is Visual Studio Code, chosen for its native TypeScript support and the Prisma, ESLint, and GitLens extensions that cover schema editing, linting, and version control without leaving the editor.

**Backend and Server Tools.** The backend runs on Node.js via NestJS, chosen over a lighter framework like plain Express specifically because AttendanceOps has twelve distinct, dependency-ordered modules (Section 3.3) — NestJS’s built-in module system, dependency injection, and guard/decorator pattern map directly onto that architecture instead of requiring it to be hand-rolled. Prisma is the ORM layer, giving the project a single schema file as the source of truth (Section 3.4) and type-safe queries generated directly from it. Redis backs two distinct needs: atomic plan-limit counters (Section 3.4, Subscription cluster) and BullMQ, the job queue used for anything that must never block a request — CSV student imports and async fraud scoring being the two cases that matter most (Section 3.3).

### 2.6.2 Programming Languages and Frameworks Used

**TypeScript** is the language across the entire stack — frontend and backend — which keeps a single mental model and a shared type layer (packages/shared-types in the monorepo) between the two. This matters more here than in a typical CRUD app: permission action strings, attendance status enums, and ticket state values all

need to match exactly between what the frontend renders and what the backend enforces, and a shared, type-checked source for those values catches drift at compile time instead of at runtime in production.

**NestJS** structures the backend into the module map described in Section 3.3. Its decorator-based guard system is what makes the “every route checks a permission string against `PERMISSION.MATRIX.md`” rule enforceable in code rather than just in documentation — a `@RequirePermission(action)` decorator on a controller method is a direct, auditable link back to a specific row in the permission matrix.

**Prisma** generates a fully typed client from `schema.prisma`, and its migration system (`prisma migrate dev`) is the only way schema changes reach the database — migrations are never hand-edited after being applied to a shared environment, per the project’s schema guide.

**React** (via Next.js) drives the frontend, structured around the four role-scoped dashboards the PRD calls for (Admin, Mentor, CR/Batch, Student). Each dashboard only renders the widgets and actions relevant to that role’s scope, which keeps the UI from becoming a single interface that tries to serve every permission level at once.

**BullMQ** runs on top of Redis as the queue for anything that has to happen asynchronously and reliably — fraud scoring after an attendance submission, and row-by-row validation during a CSV student import. Both jobs report progress back per-item, which is what lets a failed import surface as “47 succeeded, 3 failed, here’s why” instead of a single opaque failure.

**Docker Compose** runs local Postgres 16 and Redis 7, so every contributor develops against identical versions without installing either directly on their machine — this is the same infrastructure the onboarding guide targets for a sub-20-minute clone-to-running setup.

### 2.6.3 Implementation Details by Module

**Auth Module.** Handles signup (auto-creating an Organization and an ADMIN Membership in the same transaction), login, JWT issuance, and session context resolution. The active organization and effective permissions are embedded directly in the token payload, so every subsequent request carries its own scope without a separate lookup — the permission-check function still re-verifies against Membership

on every guarded action rather than trusting the token blindly, since role and status can change mid-session.

**Subscription Module.** Every plan-limit check in the system — batch creation, student import, employee invite — routes through one `checkLimit(orgId, resource)` method backed by a Redis atomic counter, rather than each module running its own `count()-then-create()` query. This is the single place where a race condition at the plan boundary can be closed, instead of needing the same fix duplicated (and potentially done inconsistently) in three different modules.

**Session Module.** Owns the rotating attendance-code generation and rotation, and is the only module permitted to write `SessionActivityEvent` rows for lifecycle transitions. The rotation interval and code length are configuration values, not hardcoded constants, so the fraud-resistance/usability tradeoff can be tuned per deployment without a code change.

**Attendance Module.** Enforces the one-submission-per-student-per-session rule at the application layer first, returning a clean “already submitted” error before the request ever reaches the database’s unique constraint — the database constraint (Section 3.4) exists as a backstop, not the primary user-facing error path.

**Fraud Module.** Runs as a BullMQ consumer, never inline in the attendance-submission request/response cycle. Risk scoring reads passively captured signals (IP, device fingerprint, timing) and writes a score plus an `isFlagged` boolean back to the Attendance row — it never blocks, delays, or rejects the original submission.

## 2.6.4 Integration of Modules

The module boundaries described architecturally in Section 3.3 are enforced in code through NestJS’s own module import system — a module can only call into another module’s exposed service methods, never reach into its Prisma models directly. This is what makes rules like “only `OrgModule` writes to `Membership`” and “only `BatchModule` writes to `BatchMembership`” actually hold at runtime rather than being a convention teammates might quietly violate under deadline pressure.

Auth integrates with every other module through a shared guard rather than being called directly — a controller method declares the permission it needs, and the guard resolves it against the current request’s `Membership` context before the method body ever runs. Session and Attendance integrate tightly (window state gates submission eligibility) but remain separate modules specifically so Session’s



lifecycle logic doesn't get tangled with Attendance's fraud and duplicate-prevention logic. Reports integrates as a pure consumer, reading from Attendance, Session, Ticket, and Fraud without ever writing back — which is what keeps it safe to eventually point at a database read replica without touching any other module.

### 2.6.5 API / External Library Integration

**BullMQ (Job Queues)** — Backs both the fraud-scoring pipeline and the CSV student-import pipeline. Configured with retry policies per job type, since a transient failure (a flaky external call, a database timeout) shouldn't require re-uploading an entire CSV or re-triggering fraud scoring for every attendance record in a session.

**JWT & Bcrypt (Security)** — Tenant authentication and Platform Admin authentication use separate JWT secrets and audiences, issued through entirely separate login endpoints, so the two identity systems can never be confused for one another at the token-verification layer. Bcrypt hashes every password before it reaches the database.

**Prisma Client (Data Access)** — Generated directly from `schema.prisma`, giving every query full type inference against the actual schema, which catches a large class of "field renamed but query not updated" bugs at compile time rather than at runtime.

**Redis (Atomic Counters & Queue Backing)** — Two distinct uses under one dependency: atomic increment/decrement operations for plan-limit enforcement (Section 3.4), and as the backing store for BullMQ's job queues. Kept as a single Redis instance in development (via Docker Compose) with the option to split into separate instances in production if the queue and counter workloads ever start contending with each other.

## 2.7 Challenges Faced During Implementation

Throughout the development of the Job Recommendation System, several technical and non-technical problems were faced. One significant technical issue was to build a good job recommendation logic with little user interaction data in the beginning. We alleviated this by initially building a rule based ranking system on users skills and job, with later intention to bring in machine learning models given more data.

Another challenge involved managing and storing terabytes of user interaction data (while keeping it fast), which was tackled with good database design, as well as indexing. Securing the authentication and role-based access control was also challenging to design and test. On a non-technical end, clarifying the requirements and reaching feature scope in a limited time and resources had been hard; this was tackled through planning, prioritizing work on tasks and iterating development. In addition, usability for different kinds of users was managed by realizing a simple interface, which is easy to use and understood following existing job portal workflows.

## 2.8 Engineering Considerations

Introduction Engineering considerations are essential in a successful design, development and implementation of the Job Recommendation System. Firstly, the system should have good scalability, performance, reliability and maintainability to support the increase of users and job entries as well as interaction data without degrading performance. The system is implemented as a set of modular components, including job management and user authentication, among others (Fig 1) all of which can be developed, tested and updated separately. The performance in data processing and storage is believed to contribute towards real-time job recommendations and analytics. Security and privacy are also put in the fore to safeguard the anonymity of sensitive user information and other trust features. Also, special focus is put on usability and accessibility to allow job seekers and employers to easily use the OWL in a minimum of training time. Documentation, versioning and testing best practices are also taken into account to help maintaining over time and easily improve in the future.

### 2.8.1 Societal Impacts of Engineering Solutions

The context of the connections between engineers and society is established by what engineering does to people. Engineers need to be aware of the safety, accessibility and welfare of those influenced by their work; and its designs must also serve the greater common good. Contemporary engineering education has a strong focus on social responsibility, and professionals are urged to reconcile the inclusive, moral with the ecologically sustainable. By being aware of these ramifications, engineers

add to technological progress in-explicably Also work toward the betterment of everyone's quality of life.

### **2.8.2 Environment and Sustainability Considerations**

We previously mentioned the environmental and sustainability considerations in the section 2, in the scenario of JRS the focus is around efficient design of a platform that consumes as little server resource or energy (to process data) as possible. The system further promotes sustainable usage of digital resources by achieving scalability optimisation and minimising computational waste. Moreover, in helping job seekers to more rapidly secure suitable employment, this platform works indirectly in the cause of social and economic sustainability by providing support for workforce development without unnecessary burdening of existing infrastructure. By incorporating sustainable engineering into this project, the system will satisfy the current user requirements and remain an effective, maintainable solution as it scales.

### **2.8.3 Ethical Considerations**

In the Job Recommendation System, ethical concerns revolve around developers' moral and professional obligations when dealing with user data and interactions. The system should also guarantee the privacy, fairness and transparency for recommendation but without either bias or manipulation towards matching jobs. Developers are expected to safeguard users' data, give them the right job recommendations and contribute honestly to its overall functioning. Basically, the implementation of this system should take into account all the aspects mentioned in to establish a publicly accepted fair and sustainable solution for workers as well as employers.

## 3. Testing and Results Analysis

### 3.1 Testing

#### 3.1.1 Testing Strategies

Testing follows a layered approach matching the module boundaries from Section 3.3. Unit tests target the permission-resolution function and the attendance-submission uniqueness/window logic first — per the project’s contribution guidelines, these two are the places a silent bug does the most damage, since a broken permission check leaks data across organizations and a broken window check breaks the core fraud-prevention promise. Integration tests run against the actual Docker Postgres/Redis stack (Section 3.7) to confirm plan-limit atomic counters and duplicate-submission constraints hold under concurrent requests, not just in isolation. System testing covers full request/response cycles through the API contract (Section 3.7.5) — attendance submission, ticket appeal resolution, fraud review — and Automation testing via CI runs the full suite (`pnpm test`, `pnpm lint`) on every pull request, per the contribution guidelines’ merge checklist, so a broken build can never reach `main`.

### 3.1.1.1 Test Case Scenarios

| TC ID | Module      | Test Case Description                                                    | Expected Result                                                                    | Status  |
|-------|-------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------|---------|
| TC01  | Auth        | Signup creates User + Organization + ADMIN Membership in one transaction | All three records exist and are linked correctly                                   | Planned |
| TC02  | Auth        | Login with valid credentials                                             | JWT issued with correct activeOrganizationId and effectivePermissions              | Planned |
| TC03  | Auth        | Login with invalid password                                              | 401, no token issued                                                               | Planned |
| TC04  | Permissions | Mentor attempts an action outside their assigned batch                   | 403, action denied despite correct role                                            | Planned |
| TC05  | Permissions | Admin acts on a batch with no explicit Batch-Membership row              | Action succeeds via implicit Admin access                                          | Planned |
| TC06  | Batch       | Create batch at plan limit                                               | 409 PLAN_LIMIT_REACHED                                                             | Planned |
| TC07  | Batch       | Two simultaneous batch-creation requests near plan cap                   | Only one succeeds; Redis counter prevents race condition                           | Planned |
| TC08  | Session     | CR opens attendance window                                               | Window opens, first rotating code generated, SessionActivityEvent logged           | Planned |
| TC09  | Attendance  | Student submits valid code within window                                 | Status derived correctly (Present/Late)                                            | Planned |
| TC10  | Attendance  | Student submits a second time for the same session                       | 409 ATTENDANCE_ALREADY_SUBMITTED                                                   | Planned |
| TC11  | Attendance  | Student submits after window closes                                      | 400 ATTENDANCE_WINDOW_CLOSED                                                       | Planned |
| TC12  | Attendance  | CR manually marks a student without providing a reason                   | Request rejected — reason is mandatory                                             | Planned |
| TC13  | Fraud       | Same device submits attendance for two different students                | Flagged for review, riskScore set, submission not blocked                          | Planned |
| TC14  | Ticket      | Student raises a ticket after session end<br>37                          | Rejected — tickets only allowed before session ends                                | Planned |
| TC15  | Ticket      | CR rejects, student appeals, Mentor accepts                              | Attendance status updates to INFORMED in the same transaction as the ticket update | Planned |
| TC16  | Audit       | Any manual attendance edit                                               | AuditLog entry created with actor reason be-                                       | Planned |

### 3.1.1.2 Performance Testing (Target Benchmarks)

Performance targets are drawn directly from the PRD’s non-functional requirements (Section 3.1.2) and success metrics, to be validated once the core attendance flow is implemented:

| Function                    | Target                                    | Result         |
|-----------------------------|-------------------------------------------|----------------|
| Attendance submission       | Under 2 minutes end-to-end for a student  | To be measured |
| General action latency      | Under 2 seconds under normal load         | To be measured |
| Concurrent load             | 200 concurrent users, no significant drop | To be measured |
| Duplicate submissions       | Zero, enforced at DB level                | To be measured |
| Background job failure rate | Under 1%                                  | To be measured |

### 3.1.2 Comparison with Initial Requirements

At the current stage, the schema (Section 3.4), permission matrix, API contract, and service-layer design are complete and internally consistent — every functional requirement from Section 3.1.1 maps to a specific module, a specific set of routes, and a specific set of database tables, which is what the API contract and schema guide were built to guarantee before any controller code is written. What remains unverified is runtime behavior: whether the Redis atomic counters actually prevent the race conditions they’re designed for under real concurrent load, whether the async fraud-scoring queue keeps up under a realistic submission volume, and whether the one-tap submission flow holds its sub-2-minute target on a typical student’s connection. These are the gaps this section will be updated to close once implementation and test execution are underway.

### 3.1.3 Strengths and Weaknesses of the Design

The identity and permission model is the strongest part of the design as it stands — the multi-membership structure (Section 3.4) and the RBAC+PBAC hybrid (Section 3.2) were built specifically to avoid the two most common failure modes in multi-tenant systems: a permission check that forgets to scope by organization, and a role

model too rigid to let one person hold two roles in the same org. The audit and trust layer is similarly thorough on paper — every manual action is reason-tagged and traceable, which is the whole basis for the “provably fraud-resistant” positioning in the vision doc.

The weakest point is the same as any pre-implementation design: none of it has been tested against real concurrent load, real fraud attempts, or real messy CSV data from an actual bulk import. The heuristic fraud-scoring approach is also a known, accepted limitation for v1 — it will produce false positives at some rate that can only really be tuned once real submission data exists to tune it against.

### **3.1.4 Planned User Feedback Process**

Once a working build exists, usability testing will follow the same shape used for comparable systems: a small group of actual CRs and Mentors from a pilot batch walking through the full loop — opening a window, submitting attendance as a student, marking someone manually, resolving a ticket — with structured interviews afterward to catch friction points. Priority areas to watch for: whether the rotating code causes confusion for students on slower connections, whether the manual-mark reason requirement feels like protection or friction to CRs, and whether the fraud review queue’s volume stays manageable enough that Mentors actually engage with it instead of ignoring it.

## 4. Conclusion and Future Work

### 4.1 Summary of Design Work Completed

At this stage, AttendanceOps has a complete design foundation rather than a finished, tested product — worth stating plainly, since the sections above describe what the system is built to do, not what has been verified running. The identity and membership model (Section 3.1) is fully specified: a single global User identity with organization-scoped Membership rows, a role set collapsed to Admin/Mentor/Student for v1 with an explicit, additive path back to a full Owner/Admin/Manager split later. The permission model (RBAC combined with scope-based checks) is documented action-by-action in the Permission Matrix, and every one of the 86 API routes in the contract is mapped to a specific permission string, so there is no ambiguity left for implementation to resolve on its own.

The database schema (Section 3.4) is complete down to the field level, with cascade and restrict rules chosen deliberately rather than left at Prisma's defaults — the Attendance → BatchMembership restrict rule in particular is what makes "you can never accidentally delete a student's attendance history" a database-enforced guarantee rather than a hope. The service-layer design (Section 3.3) establishes a one-writer-per-table rule across twelve modules, which is what keeps a system this permission-heavy from turning into a tangle of modules reaching into each other's data.

#### 4.1.1 Fulfillment of Project Objectives

Measured against the specific objectives set out in Section 1.4.2:

**Identity and multi-org membership** — fully designed, including the less obvious cases (a student promoted to Mentor, the same email invited to a second org,



an Admin who also directly mentors one batch) that a simpler role model would have missed.

**Rotating attendance code** — designed to defeat screenshot-sharing without adding a second step for the student, with the exact rotation mechanism (interval, code length) left as a tunable configuration value rather than hardcoded.

**Manual marking and ticket/appeal system** — the full state machine is specified, including the transactional guarantee that a ticket's resolution and the underlying attendance status update happen atomically, so a partial failure can never leave the two inconsistent.

**Passive, asynchronous fraud detection** — designed to run entirely outside the request/response cycle via BullMQ, so the "one-tap submission" promise to students is structurally protected rather than dependent on the fraud logic staying fast.

**Full audit logging** — every manual action's schema includes actor, reason, and before/after value by design, not as an afterthought bolted onto an existing table.

**Plan-based limits via atomic counters** — designed specifically to close the race-condition class of bug that count-then-create logic is vulnerable to.

**Role-scoped reporting** — the dashboard and reports structure is specified per role, though the dynamic report builder's actual query engine is design-only at this stage.

**Platform Admin isolation** — fully designed as a separate auth audience, separate audit log, and separate identity model from tenant accounts, which was one of the harder architectural decisions to get right given how much of the rest of the system assumes a single Membership-based identity.

None of these are yet validated against a running system, which is the honest gap between "designed to" and "does."

#### 4.1.2 Limitations of the Current Design

The heuristic fraud-scoring approach is an accepted v1 limitation — without real submission data, there is no way to know its actual false-positive rate, and it will need tuning once a pilot batch generates real traffic to tune against. Presence tracking via heartbeat is explicitly framed as an engagement signal rather than verified attendance, which is a deliberate scope limitation rather than an oversight, but it does mean the system cannot make claims about physical presence that some

organizations might expect from an “attendance” product. The v1 role collapse (Owner/Admin/Manager merged into a single Admin) simplifies permission logic now but means any organization needing delegated, scoped authority below full Admin has to wait for that split to be reintroduced. Finally, the entire design is unvalidated at scale — the atomic-counter and async-queue approaches are sound on paper, but their behavior under real concurrent load, and under a genuinely large CSV import, is untested.

### 4.1.3 Recommendations for Enhancement

Once Phase 1 is built and running against a real pilot org, the fraud-scoring heuristics should be revisited with actual flagged-case data before any AI-based detection is considered — replacing a working heuristic with an unvalidated model would trade a known, tunable false-positive rate for an unknown one. The dynamic report builder and full platform observability tooling (job queue monitoring, error tracking) should follow once there’s enough real usage data to make those dashboards meaningful rather than speculative. Native Meet/Zoom join-verification is worth revisiting if heartbeat-based presence tracking proves insufficient for organizations that need a stronger attendance guarantee than an engagement signal.

### 4.1.4 Future Scope of the Project

The suggested phasing carries the project from MVP to a fully observable, scaled platform:

**Phase 1 (MVP)** covers the identity/membership model, single-org operational flows across batch, session, attendance, and tickets, basic fraud prevention (rotating codes plus duplicate detection), core reports, and a minimal platform admin panel — organization list, plan assignment, and a basic health view. This is the phase that proves the core loop works end to end for one real organization before anything else is built on top of it.

**Phase 2** adds presence/heartbeat tracking, risk scoring, the dynamic report builder, full platform observability (job queue status, error tracking, cross-org abuse monitoring), and Discord webhook notifications — the layer that turns a working MVP into something a platform operator can actually run and support at more than one tenant.

**Phase 3** covers multi-batch scale features, the advanced analytics engine, impersonation tooling, the platform-wide broadcast system, and deeper billing/plan tooling — the point at which the platform is no longer just functionally complete but operationally mature enough to run as a real SaaS business across many tenants at once.

Reports and Platform Admin are deliberately sequenced last in every phase rather than built early, since building either against a real feed of attendance data is far more reliable than guessing upfront at what the dashboards need to show.

## Reference